

Informe de Arquitectura

RedNorte

Título del Proyecto: Sistema de Microservicios RedNorte

Autor: Sebastián Godoy

Fecha: Junio 2026

Institución: Instituto Profesional DuocUC

Índice

A. Diseño de Arquitectura y Patrones

A.1. Introducción y Contexto del Proyecto

A.2. Evolución de la Arquitectura

A.3. Arquitectura Propuesta

A.3.1. Capa de Presentación (Frontend)

A.3.2. Capa de Lógica de Negocio (Microservicios)

A.3.3. Capa de Persistencia

A.3.4. Capa de Seguridad y Distribución

A.3.5. Sostenibilidad a Largo Plazo

A.4. Mecanismo de Autenticación

A.5. Patrones de Diseño Aplicados

A.6. Especificaciones Técnicas

B. Entregables

B.1. Lista de Entregables

B.2. Visualización del Diagrama

C. Conclusión

A. Diseño de Arquitectura y Patrones

A.1. Introducción y Contexto del Proyecto

El Servicio Público de Salud RedNorte enfrenta una problemática operativa crítica: largas listas de espera para atención médica, sistemas de registro obsoletos que no se comunican entre sí, y una comunicación deficiente con los pacientes. El objetivo de este proyecto es el desarrollo de una plataforma digital basada en microservicios que permita centralizar la gestión de listas de espera, optimizar la asignación y reasignación de horas médicas, y proporcionar mayor visibilidad tanto al personal clínico como a los ciudadanos.

A.2. Evolución de la Arquitectura

La concepción inicial del proyecto consideraba un enfoque tradicional: migrar desde servidores físicos hacia un entorno de nube con un clúster de Kubernetes, microservicios en Spring Boot, frontend en React servido con NGINX, y PostgreSQL como motor de base de datos.

Durante la fase de validación, este modelo reveló limitaciones significativas. El primer obstáculo fue la complejidad operativa: administrar Kubernetes, RabbitMQ y volúmenes persistentes requiere tiempo y conocimientos especializados. El segundo fue el acoplamiento tecnológico: Spring Boot agregaba peso innecesario para el volumen de operaciones del sistema. El tercero fue la latencia de desarrollo: mantener un BFF tradicional implicaba escribir y mantener código que podía ser reemplazado por una capa de configuración.

Por estas razones, se rediseñó la arquitectura hacia un modelo de microservicios livianos orquestados con Docker Compose, utilizando un API Gateway (KrakenD) en lugar de un BFF tradicional, y reemplazando Spring Boot por Bun + Hono para reducir la sobrecarga de ejecución y acelerar el desarrollo.

A.3. Arquitectura Propuesta

La arquitectura final se compone de cuatro capas: presentación, API Gateway, lógica de negocio y persistencia. Todos los componentes se ejecutan como contenedores Docker orquestados con Docker Compose.

```
Frontend (Astro + Preact)
| HTTP (localhost:4321)
```

```
v
KrakenD API Gateway (puerto 8080)
| |
v v
ms-users:3001 ms-waitlist:3000
| |
v v
users_db waitlist_db
| (evento)
v
RabbitMQ
|
v
ms-notifications:3002 (WebSocket)
|
v
notifications_db
```

A.3.1. Capa de Presentación (Frontend)

El frontend se construye con Astro 4. Astro envía cero JavaScript por defecto: las páginas generadas son HTML y CSS puro, resultando en velocidades de carga muy superiores. Para los componentes que requieren interactividad, como formularios de login y tablas de lista de espera, se utiliza Preact 10 mediante el concepto de islas de interactividad. Preact ocupa aproximadamente 5 KB, frente a los 100+ KB de React. El estado global se gestiona con Nanostores (<1 KB) y los estilos con Tailwind CSS 3. El frontend se ejecuta en el puerto 4321 y se conecta al API Gateway en <http://localhost:8080/api>.

A.3.2. Capa de Lógica de Negocio (Microservicios)

La lógica de negocio se implementa mediante tres microservicios independientes que exponen una API REST. Todos están desarrollados con Hono 4 y se ejecutan sobre Bun 1.3.14.

Bun se eligió por su arranque rápido y memoria eficiente. Incluye test runner, gestor de paquetes y compilador integrados, eliminando dependencias externas.

- **ms-users** (puerto 3001): Gestiona registro, login y consulta de usuarios. Hashea contraseñas con Argon2 via Bun.password, firma JWT con jsonwebtoken.
- **ms-waitlist** (puerto 3000): Gestiona la cola de pacientes con prioridades (1-4) y estados (waiting, attending, finished, cancelled). Valida JWT mediante @hono/jwt. Publica eventos en RabbitMQ al cambiar estados.
- **ms-notifications** (puerto 3002): Escucha eventos de RabbitMQ y los entrega al usuario vía WebSocket en tiempo real. Usa Bun.serve nativo para manejar HTTP y WebSocket en el mismo puerto.

Flujo de comunicación:

1. El frontend llama a KrakenD vía HTTP
2. KrakenD reenvía a ms-users o ms-waitlist según la ruta
3. ms-waitlist publica eventos en RabbitMQ al agregar o cambiar estado
4. ms-notifications consume eventos de RabbitMQ, persiste en BD y envía por WebSocket al frontend

El frontend se conecta directamente a ms-notifications vía WebSocket (no a través de KrakenD), porque KrakenD no maneja WebSocket de manera confiable.

A.3.3. Capa de Persistencia

Cada microservicio tiene su propia base de datos PostgreSQL 16 independiente:

Microservicio	Base de Datos	Puerto	Propósito
ms-users	users_db	5432	Usuarios, roles, autenticación
ms-waitlist	waitlist_db	5433	Lista de espera médica
ms-notifications	notifications_db	5434	Notificaciones persistentes

No se utiliza ORM. El driver nativo postgres para Bun ejecuta SQL directo.

Esquema de users_db:

```
CREATE TABLE roles (  
  id TEXT PRIMARY KEY,  
  key TEXT UNIQUE NOT NULL,  
  description TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
CREATE TABLE users (  
  id TEXT PRIMARY KEY,  
  first_name TEXT NOT NULL,  
  last_name TEXT,  
  is_verified BOOLEAN DEFAULT FALSE,  
  email TEXT UNIQUE NOT NULL,  
  password TEXT NOT NULL,  
  role_id TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE RESTRICT  
);
```

Esquema de waitlist_db:

```
CREATE TABLE waitlist_entries (  
  id SERIAL PRIMARY KEY,  
  user_id TEXT NOT NULL,  
  priority INTEGER NOT NULL DEFAULT 1,  
  status TEXT NOT NULL DEFAULT 'waiting',  
  reason TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Esquema de notifications_db:

```
CREATE TABLE notifications (  
  id SERIAL PRIMARY KEY,
```

```
user_id TEXT NOT NULL,  
type VARCHAR(50) NOT NULL,  
message TEXT NOT NULL,  
is_read BOOLEAN DEFAULT FALSE,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Migraciones: Cada microservicio ejecuta migraciones al iniciar mediante `bun run src/db/migrate.ts`, con tabla de control `_migrations_history` que asegura que cada migración se aplique una sola vez.

Persistencia: Volúmenes Docker (`pgdata_users`, `pgdata_waitlist`, `notifications_data`) que persisten entre reinicios.

A.3.4. Capa de Seguridad y Distribución

- **API Gateway (KrakenD):** KrakenD es el punto de entrada único con 8 endpoints: 4 públicos hacia `ms-users` (login, registro, listar, obtener por ID) y 4 protegidos con JWT hacia `ms-waitlist` (agregar, listar, mine, actualizar estado). Pasa el header `Authorization` mediante `input_headers` y `headers_to_pass`.
- **CORS:** Solo permite `http://localhost:4321`.
- **Red Docker:** Comunicación interna en red bridge con healthchecks.
- **Aislamiento:** Cada microservicio tiene su propia base de datos, limitando el impacto de una falla.

A.3.5. Sostenibilidad a Largo Plazo

La arquitectura Docker Compose es portable: se despliega con `docker compose up --build` en cualquier entorno. Los microservicios son escalables horizontalmente. Los volúmenes Docker mantienen los datos. El proyecto está versionado en GitHub como monorepo.

A.4. Mecanismo de Autenticación

La autenticación es manual, sin servicios externos:

1. **Registro:** POST /api/auth/users con { first_name, email, password }. ms-users valida email único, hashea con Argon2 (Bun.password.hash) y genera UUID.
2. **Login:** POST /api/auth/users/login. Verifica contraseña con Bun.password.verify, firma JWT con jsonwebtoken usando JWT_SECRET.
3. **JWT:** Contiene { id, email, role_id } con expiración de 2 horas.
4. **Validación:** ms-waitlist usa middleware @hono/jwt que valida firma y expiración en cada petición. Token inválido o expirado devuelve 401.
5. **Ruta /mine:** Extrae userId del JWT en lugar de recibirlo por query param.

A.5. Patrones de Diseño Aplicados

- **Repository Pattern:** Cada microservicio encapsula consultas SQL en repositorios. El controlador y servicio nunca interactúan directamente con la BD.
- **Service Layer:** La lógica de negocio (validaciones, reglas) reside en servicios, no en controladores.
- **Controller:** Los controladores solo orquestan request y response.
- **Centralized Error Handler:** Clase AppError con statusCode. Los servicios lanzan AppError y app.onError() captura todo, devolviendo { success: false, message }.
- **Observer (RabbitMQ → WebSocket):** ms-waitlist publica eventos en RabbitMQ al cambiar estados. ms-notifications consume, persiste y envía por WebSocket.
- **Connection Manager:** Map<userId, WebSocket> que gestiona conexiones activas en ms-notifications.
- **JWT Middleware:** @hono/jwt aplicado a todas las rutas de ms-waitlist.

A.6. Especificaciones Técnicas

Componente	Tecnología	Versión
Runtime backend	Bun	1.3.14
Framework backend	Hono	4.12.x
API Gateway	KrakenD	2.13.7
Lenguaje	TypeScript	5.x

Framework frontend	Astro	4.x
Componentes frontend	Preact	10.x
Estado global	Nanostores	Última
Estilos	Tailwind CSS	3.x
Motor BD	PostgreSQL (alpine)	16
Driver BD	postgres (nativo)	3.x
Mensajería	RabbitMQ	latest
Autenticación	jsonwebtoken + @hono/jwt	Última
Hash de contraseñas	Bun.password (Argon2)	Nativo
Testing backend	Bun test	Nativo
Testing frontend	Vitest + happy-dom	4.x
Cobertura	@vitest/coverage-v8	Última
Orquestación	Docker Compose	Última

B. Entregables

B.1. Lista de Entregables

1. Código fuente (monorepo GitHub): <https://github.com/GoguX76/rednorte>
2. Microservicios:
 - backend/ms-users/ — usuarios (registro, login, JWT)
 - backend/ms-waitlist/ — lista de espera (CRUD + RabbitMQ)
 - backend/ms-notifications/ — notificaciones (WebSocket + RabbitMQ)
3. API Gateway: backend/bff/krakend.json
4. Frontend: frontend/ (Astro 4 + Preact 10)
5. Diagramas de arquitectura:
 - C1: docs/diagrams/rednorte-c1.png
 - C2: docs/diagrams/rednorte-c2.png
 - C3: docs/diagrams/rednorte-c3.html
6. Documentación de persistencia: docs/data-persistence.md
7. Informe de pruebas unitarias: docs/unit-test-report.md (94 tests, $\geq 94\%$ cobertura)
8. Postman Collection: docs/postman-collection.json (8 endpoints)
9. README.md en cada microservicio y frontend con instrucciones + tabla técnica

B.2. Visualización del Diagrama

Nivel	Archivo	Descripción
C1	rednorte-c1.png	Contexto: actores y sistema RedNorte
C2	rednorte-c2.png	Contenedores: frontend, KrakenD, 3 microservicios, RabbitMQ, PostgreSQL
C3	rednorte-c3.html	Componentes internos: routes, controllers, services, repositories, middleware

C. Conclusión

La arquitectura implementada para RedNorte representa una solución robusta y desacoplada basada en microservicios. La adopción de KrakenD como API Gateway eliminó la necesidad de mantener un BFF tradicional, reduciendo el código a configurar y mejorando el rendimiento. Los tres microservicios están aislados con sus propias bases de datos PostgreSQL, siguiendo el principio de responsabilidad única.

La integración de RabbitMQ como bus de eventos permite la comunicación asíncrona entre ms-waitlist y ms-notifications, garantizando que las notificaciones se entreguen incluso si el servicio receptor está temporalmente caído. Las notificaciones en tiempo real llegan al frontend mediante WebSocket nativo de Bun, con reconexión automática y backoff exponencial.

El frontend con Astro y Preact ofrece tiempos de carga mínimos al enviar cero JavaScript por defecto. La cobertura de pruebas unitarias supera el 94% en los cuatro componentes del sistema, con 96 pruebas automatizadas. Los patrones Repository, Service Layer, Controller y Centralized Error Handler mantienen el código mantenible y testeable.

Finalmente, Docker Compose permite desplegar el sistema completo con un solo comando en cualquier entorno que soporte Docker, y la estructura monorepo en GitHub facilita el versionado y la colaboración.